FIRMWARE FUNCTIONALITY

OUTPUT:
The output format of the firmware would be a **JSON file**. The below table consists of the required metadata that will be sent to the client **provided API**.

Sno	Parameter	Format	Description
1	device_id	Long Integer	Unique identifier of the tag
2	timestamp	yyyy:mm:dd hh:mm:ss	Timestamp at which that particular data packet was recorded
3	Battery level percentage	Float	Battery life remaining of the asset
4	IMU Sensor Data	Object: Dictionary style	This object consists of the fused data collected by IMU sensor
5	GPS Sensor data	Object: Dictionary style	This object consists of the fused coordinates collected by GPS module

LED FUNCTIONALITY:
We have currently 6 LED's in place to describe the state and status of the asset tag.

1. **Battery Charging:** 2 LED's are used to indicate the Battery level. One to indicate its charging status and one to indicate that the battery is fully charged.
2. **Battery Life Indication:** 1 LED will be used to indicate the battery level remaining i.e [high, medium, low] will be respectively [Green, Yellow, Red]

Sno	Battery Life Remaining	LED Color
1	High	Green
2	Medium	Yellow
3	Low	Red

3. **Device Status:** 1 LED will be used to describe the status of the asset tag.

Sno	State	LED Color
1	Bad	RED
2	Good	GREEN

4. **Wifi Transmission:** 1 LED will be used to describe the wifi transmission status

Sno	State	LED Color
1	Default	OFF
2	Transmitting	Blinking

5. **BLE Transmission:** 1 LED will be used to describe the BLE transmission status

Sno	State	LED Color
1	Default	OFF
2	Transmitting	Blinking

EDGE CASE HANDLING:(Optional)

CASE 1: WiFi Transmission Failure
In case the **asset tag fails** to send data through an API due to **network failure**, **API failure etc**. We store the failed transmission data and append it to the next transmitted data. If the transmission fails consequently for **2 times,** We enable the **BLE (Bluetooth Low Energy)** as fallback to send the failed transmission data in the form of **BLE protocol (User Datagram Protocol)**.

CASE 2: Bluetooth Module
To accept the **data** sent through **BLE** we need a user interface that can connect to the modules and listen on PORT. Once configured with the required listener port and receiver port.
A CRON could be set up from the client side where they can push this data to the API on a daily basis to ensure no data is lost.

WIFI CREDENTIALS UPDATE:

Over BLE
We code the firmware in such a way that a user can update the wifi credentials of the asset tag by connecting to the **module over bluetooth (BLE)** over an app or web interface developed. We need this initial setup in place and ensure that the WiFI SSID and password remain

unchanged during this point of time. In case of any change, there would be a need to update the firmware again with wifi credentials.

BATTERY OPTIMISATION:
Need to be in low power modes to extend battery life such that module performs for a longer life with the same efficiency and accuracy.

NOTE TO KEEP IN MIND:
N4R2 variant needs to be considered.